MNIST Classification: Overfitting and Regularization Using CNN

PyTorch CNN - Full MNIST Dataset, Stratified 70/15/15 Split

https://github.com/Abel-Jacob/MNIST-overfitting-analysis

Abel Jacob

Introduction

The objective of this project was to study how neural networks overfit on training data and how regularization techniques can improve their generalization performance. The model was implemented using **PyTorch**, with supporting libraries including **Torchvision** for dataset loading and preprocessing, **NumPy** for numerical operations, **Matplotlib** for visualization, and **Scikit-learn** for evaluation metrics.

The **MNIST handwritten digit dataset** was used to train and evaluate a Convolutional Neural Network (CNN). To analyze overfitting, the project was divided into three experiments:

1. **Baseline:** A small, appropriately-sized CNN was trained with no regularization and no excess capacity, to confirm that MNIST generalizes cleanly when model capacity is matched to the task.
2. **Part A:** A large CNN was trained without any regularization, allowing the model to memorize the training data and demonstrate overfitting.
3. **Part B:** The same CNN architecture was trained using regularization techniques such as data augmentation, weight decay, early stopping, and learning rate scheduling to improve generalization.

Finally, the performance of all three models was compared using training and validation curves, test accuracy, and confusion matrices.

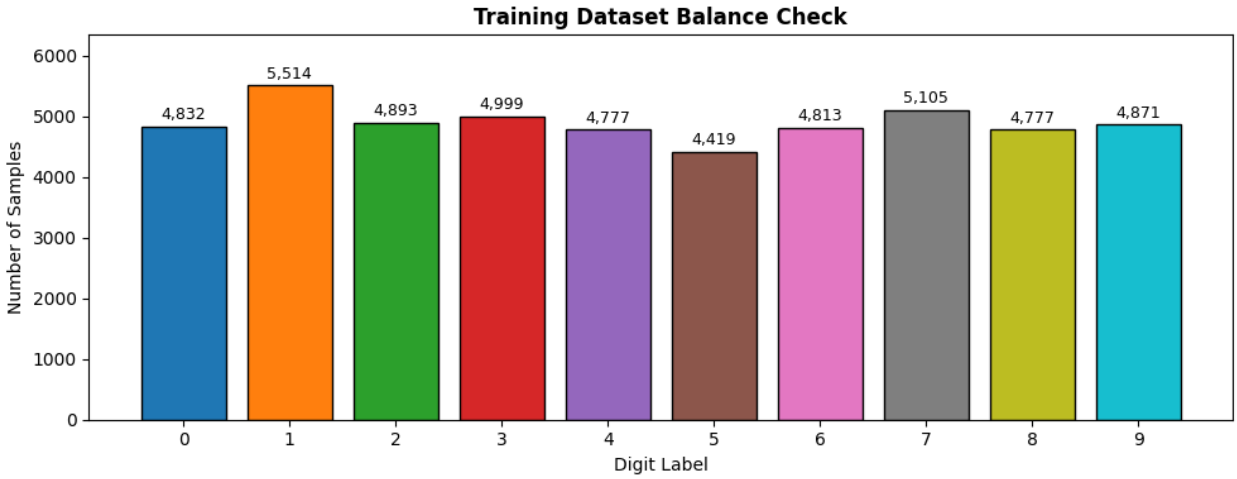
Dataset and Preprocessing

I combined all 70,000 samples (60,000 training and 10,000 test) into a single pool in order to create the dataset splits. I used a layered split to separate the pictures from this pool into:

- **70%** for training (49,000 images)
- **15%** for validation (10,500 images)
- **15%** for testing (10,500 images)

The stratified split ensures that each of the three subsets maintains an identical proportion of the 10-digit classes. I used the standard MNIST mean (0.1307) and standard deviation (0.3081) to normalize every image.

A sample of 20 training images with their ground-truth labels. These are raw, unaugmented digits straight from the training split, used as a sanity check that labels line up correctly with images before any training begins.



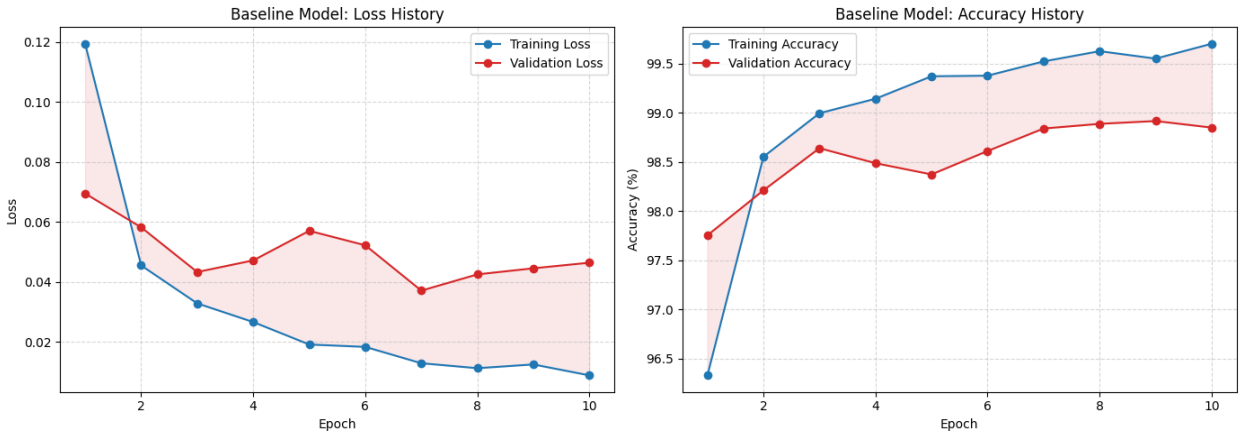
Class distribution across the training split, ranging from 4,419 to 5,514 samples per digit. The stratified split keeps this close to the natural distribution of digits in the full MNIST dataset, so no single class is over- or under-represented during training.

Baseline Model

Before deliberately pushing the network into overfitting territory, it helps to establish a baseline: a small, appropriately-sized CNN trained with no regularization tricks and no excess capacity. This serves a critical role in the study, since if a well-matched model generalizes cleanly on its own, it confirms that the overfitting observed in Part A is a direct consequence of overparameterization, not some inherent property of the dataset or the training setup.

The baseline uses two convolutional blocks (32 and 64 filters respectively), each followed by batch normalization and max pooling, feeding into a compact two-layer classifier head. Batch

normalization is included here as standard training practice, not as a regularizer, to keep activations stable during the 10-epoch run. No dropout, weight decay, early stopping, or data augmentation is used. At roughly 421,834 parameters, about one-tenth of the ExperimentalCNN used in Parts A and B, this architecture is appropriately matched to 28x28 greyscale digit classification.



By the end of the 10-epoch run, the baseline model reached **99.70%** training accuracy against **98.85%** validation accuracy, a gap of **0.85%,** with a validation loss **5.27** times the training loss (0.0464 vs. 0.0088). On the held-out test set it achieved **98.85%** test accuracy with a test loss of **0.0426**, only a **0.85%** train-to-test gap. As expected, matching model capacity to task complexity keeps generalization tight without any explicit regularization, confirming that the overfitting seen in Part A is a direct consequence of the ExperimentalCNN's excess capacity rather than a property of the dataset itself.

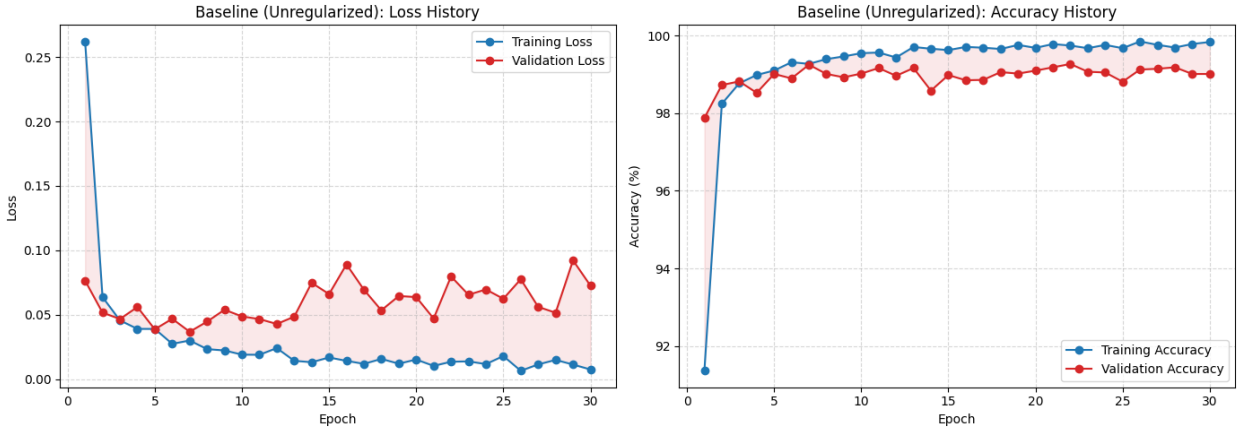
Model Architecture: Designed to Overfit

I constructed a large Convolutional Neural Network with six convolutional layers arranged in three blocks (64, 128, and 256 filters, respectively) and four fully connected layers (1024 → 512 → 256 → 10 units) to force the model to overfit. This model has more than 4.16 million trainable parameters, totaling 4,163,274, or about 85 parameters for each training image. This is far more capacity than 28x28 greyscale digits actually require.

By default, I did not include any dropout or batch normalization layers, because the point of this first run was to watch the model start memorizing the data unconstrained.

Part A: Training Without Any Regularization

I trained the network for thirty epochs in this experiment. I did not use weight decay, and the training images were loaded without any data augmentation transforms. Running the model for 30 epochs lets us observe the generalization gap develop as training continued.



The model's final validation loss was approximately **9.80** times more than the training loss (0.0725 vs. 0.0074), and by the end of training, it had achieved **99.83%** training accuracy against **99.01%** validation accuracy and a gap of **0.82%.**

That loss ratio is really the more telling number here: the accuracy gap looks modest, but the model is far less confident and far less consistent on validation data than the training-accuracy figure alone would suggest.

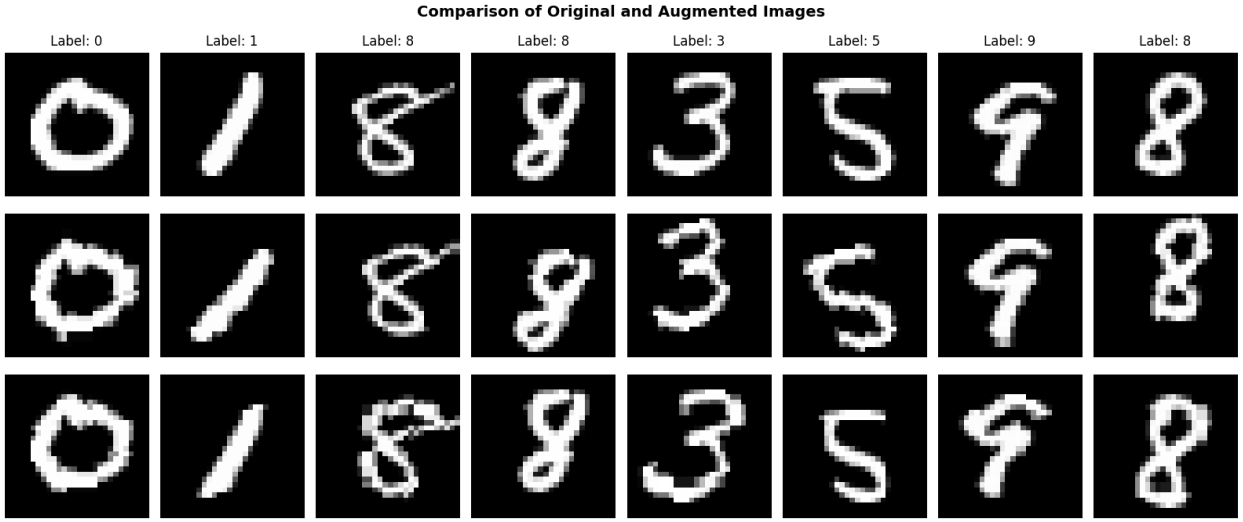
Why the Gap Isn't Larger Than Expected: CNN Inductive Bias

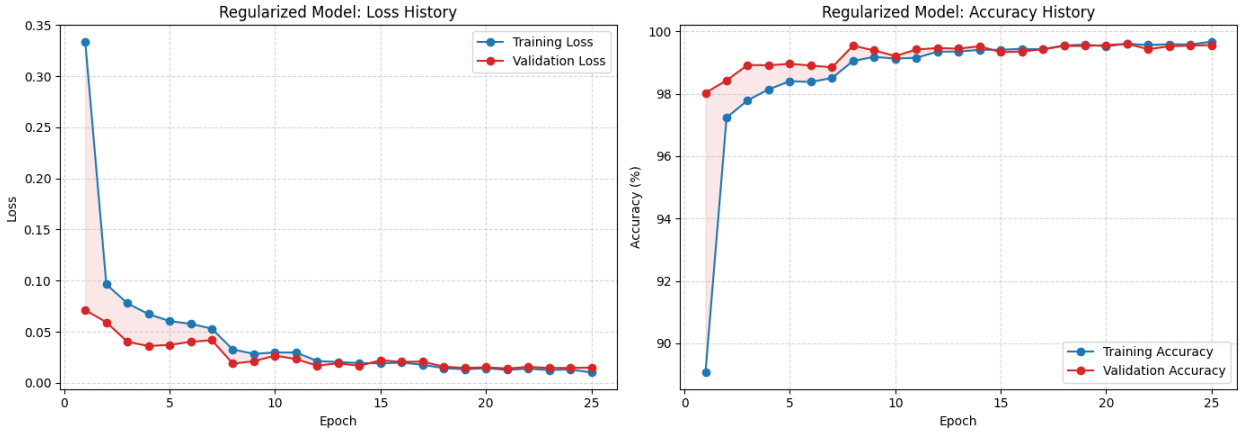
Although the CNN was designed to overfit, it still generalized reasonably well because of its built-in architectural properties. **Weight sharing** allows the same filters to detect features across the entire image, **spatial locality** enables the model to learn local patterns such as edges and curves, and **translation equivariance** helps recognize features even when they appear in different positions. These properties act as implicit regularization, making CNNs naturally more resistant to overfitting than dense networks. Consequently, the unregularized CNN showed only a small generalization gap despite its high capacity.

Part B: Training with Regularization

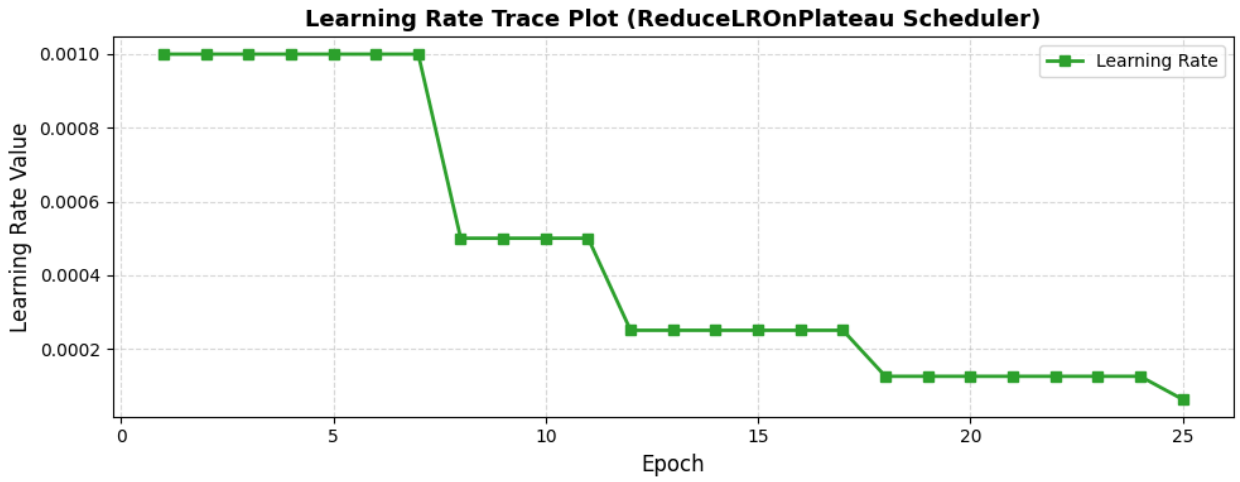
To stop the model from memorizing the data, I trained the **same CNN architecture** using the following regularization techniques:

1. **Data Augmentation:** To improve the diversity of training data, random rotations, translations, and scaling were applied.
2. **Weight Decay:** To avoid too high weight values, an L2 penalty (1e-4) was included.
3. **Early Stopping:** Stopped training when the validation loss stopped improving for four consecutive epochs.
4. **Learning Rate Scheduling:** Reduced the learning rate by half when validation loss plateaued for two epochs.



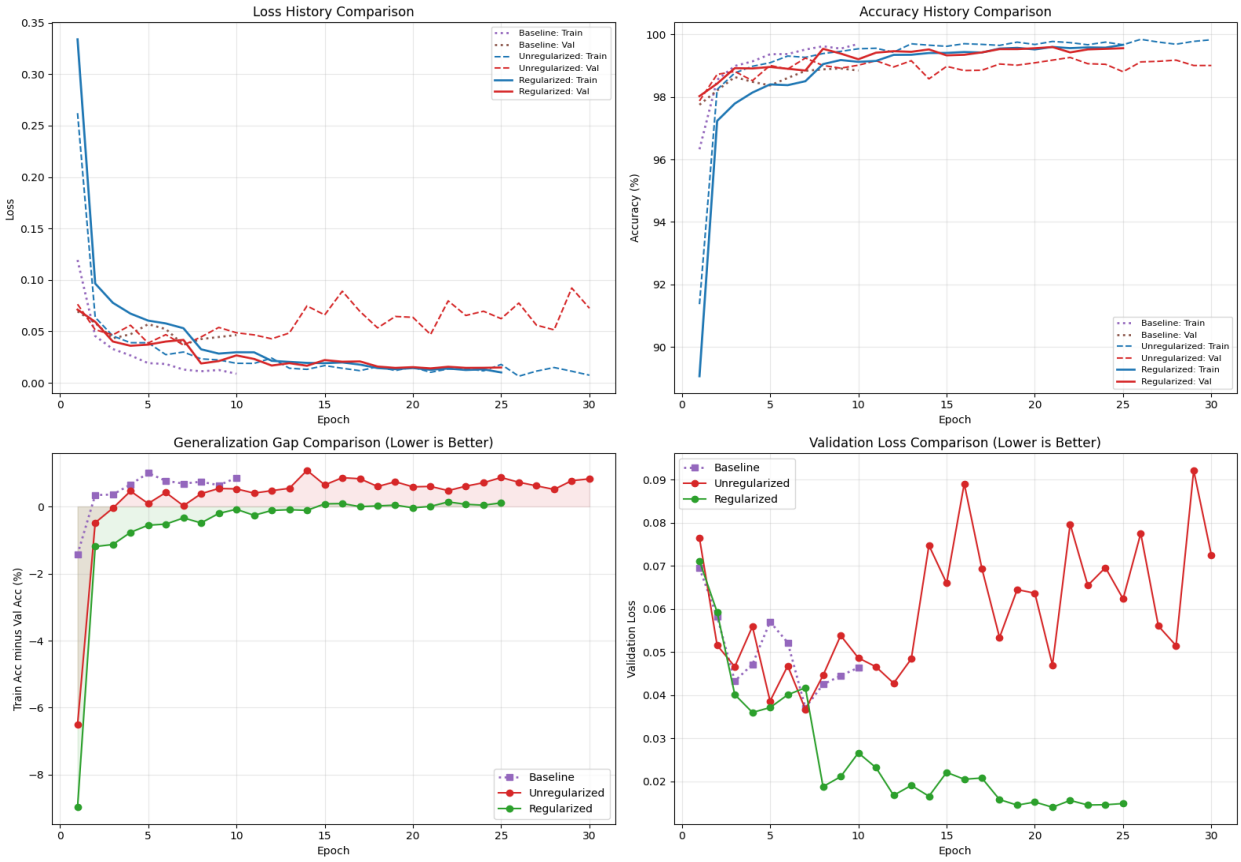


By the end of training, the regularized model reached **99.67% training accuracy** against **99.56% validation accuracy** and a gap of just **0.11%,** with validation loss only 1.48 times the training loss (0.0149 vs. 0.0101). Both the accuracy gap and loss ratio dropped sharply compared to Part A.



The ReduceLROnPlateau scheduler modifies the learning rate during training. For the first seven epochs, the learning rate stays at 0.001. The scheduler gradually halves the learning rate to 0.0005 (about **epoch 8**), 0.00025 (about **epoch 12**), 0.000125 (about **epoch 18**), and 0.0000625 (about **epoch 25**) as the validation loss plateaus.

Comparing all three runs



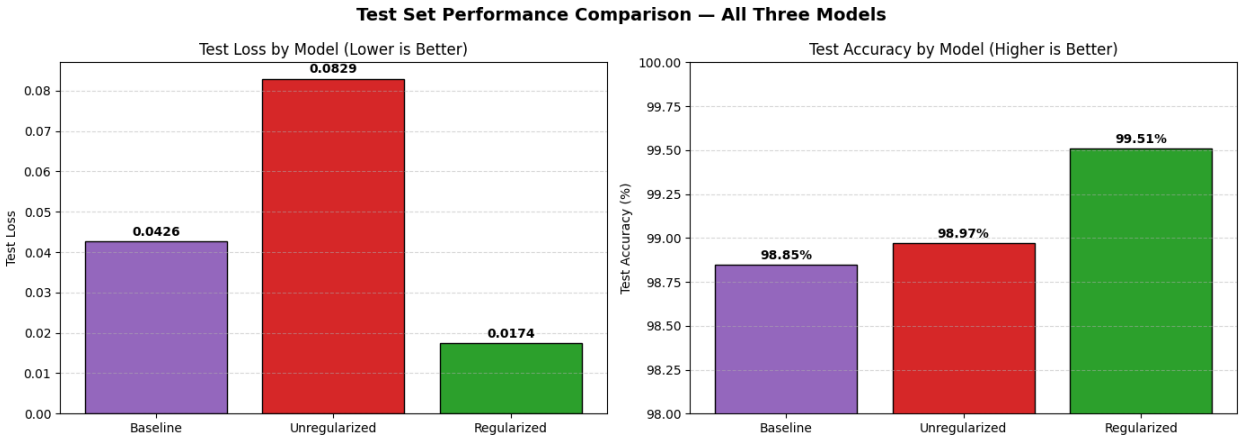
Generalization Gap

The generalization gap represents the difference between training and validation accuracy. In the unregularized model, the gap increased over time, indicating that the model was memorizing the training data. In contrast, the regularized model maintained a small and stable gap, showing that it generalized better to unseen data. A smaller gap indicates reduced overfitting and better overall model performance.

Test Set Performance and Error Analysis

With all three models trained, I evaluated each on the held-out 10,500-image test set which is data none of the models had seen during training or validation.

Model	Test Accuracy	Test Loss	Train → Test Gap
Baseline Model	98.85%	0.0426	0.85%
Unregularized Model	98.97%	0.0829	0.86%
Regularized Model	99.51%	0.0174	0.16%

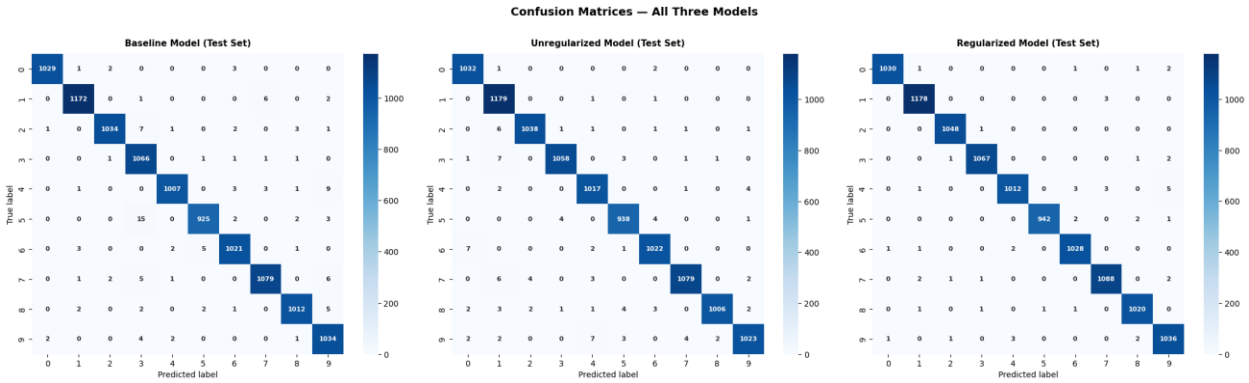


The regularized model comes out ahead on every measure here: higher test accuracy, less than a third of the test loss, and a train-to-test accuracy gap roughly four times smaller. The full picture across all three splits is summarized below.

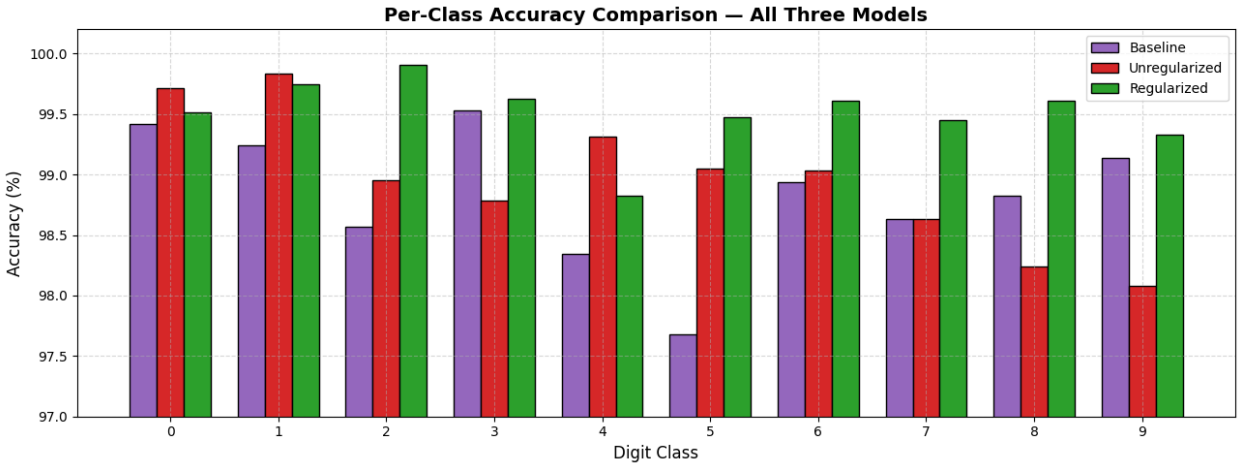
Summary Comparison Table

Model	Train Acc	Val Acc	Test Acc	Train–Val Gap	Test Loss
Baseline	99.70%	98.85%	98.85%	0.85%	0.0426
Unregularized	99.83%	99.01%	98.97%	0.82%	0.0829
Regularized	99.67%	99.56%	99.51%	0.11%	0.0174

Confusion Matrices



Confusion matrices for all three models on the held-out test set, with true labels on the y-axis and predicted labels on the x-axis. All three matrices are heavily diagonal, as expected for models scoring at or above 99% test accuracy, with the regularized model (right) showing the fewest scattered off-diagonal errors overall.



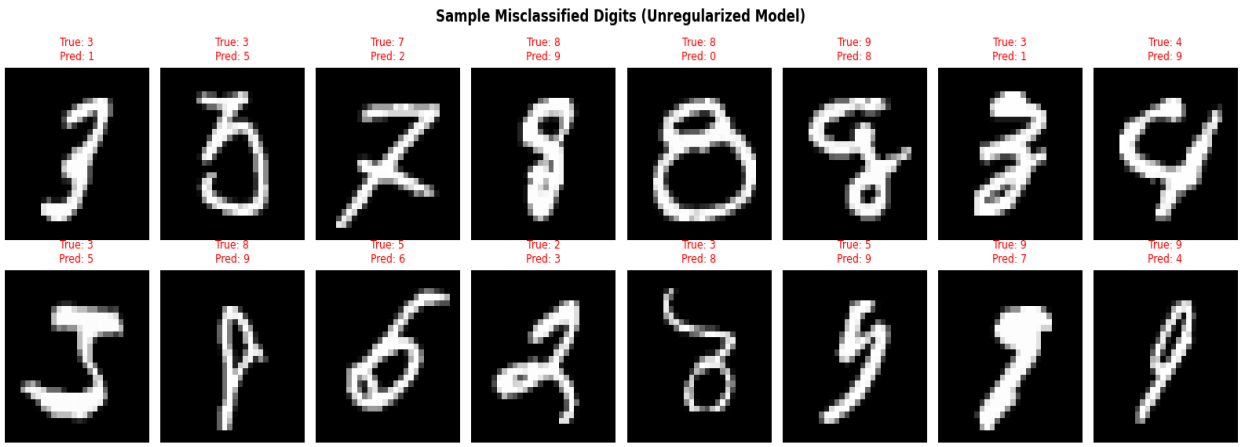
Detailed Classification Reports

Precision, recall, and F1-score for every digit, for all three models, on the test set:

Digit	Base. P	Base. R	Base. F1	Unreg. P	Unreg. R	Unreg. F1	Reg. P	Reg. R	Reg. F1
0	1.00	0.99	1.00	0.99	1.00	1.00	1.00	1.00	1.00
1	0.99	0.99	0.99	0.99	0.99	0.99	1.00	1.00	1.00
2	1.00	0.99	0.99	1.00	0.99	0.99	1.00	1.00	1.00
3	0.97	1.00	0.98	0.99	1.00	0.99	1.00	0.99	0.99
4	0.99	0.98	0.99	0.99	0.99	0.99	0.99	0.99	0.99
5	0.99	0.98	0.98	1.00	0.99	0.99	0.99	1.00	0.99
6	0.99	0.99	0.99	0.99	0.99	0.99	1.00	0.99	0.99
7	0.99	0.99	0.99	0.99	1.00	0.99	0.99	0.99	0.99
8	0.99	0.99	0.99	0.99	0.99	0.99	1.00	0.99	0.99
9	0.98	0.99	0.98	0.99	0.99	0.99	1.00	0.99	0.99
Accuracy						0.99			0.99
Macro avg	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99
Weighted avg	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.99

Both models perform within a hair of each other on a per-class basis at this level of accuracy, the differences are mostly in the third decimal place. The regularized model edges out the unregularized one on precision for several digits (0, 1, 2, 6, 8, 9), which lines up with its cleaner confusion matrix.

Misclassified Samples



A sample of digits the unregularized model misclassified on the test set, with the true label and the model's (incorrect) prediction shown above each image. Many of these are genuinely ambiguous handwriting samples, digits that even a human reader might hesitate over rather than cases where the model is making an obviously unreasonable guess.

Conclusion

This experiment showed that the training procedure, not only the model's capabilities, is what causes overfitting. A small, appropriately-sized baseline CNN (421K parameters) confirmed that MNIST generalizes cleanly when capacity is matched to the task, reaching 98.85% test accuracy with a near-zero 0.85% train-val gap. Because of the implicit regularization offered by the convolutional architecture, the unregularized CNN only displayed a modest accuracy gap of 0.82% despite having 4.16 million parameters, roughly ten times the baseline's size.

However, its validation loss was 9.80 times its training loss, indicating true memorization that accuracy alone was unable to fully capture. Test accuracy increased from 98.97% to 99.51% when data augmentation, weight decay, learning rate scheduling, and early stopping were applied to the same architecture, reducing the accuracy difference to 0.11% and the loss ratio to 1.48x, matching the baseline's own generalization behavior despite the model's far greater capacity. This demonstrates that by limiting the training process alone, the same model capacity that leads to overfitting can be successfully made to generalize.

***